Lab 2 — Password Cracking (Windows, Linux & ZIP)

Theme: *Red vs Blue* — Can you break it, can you fix it?

Time: 2–3 hours

DUE DATE: October 10, 2026

Tools Needed:

- John the Ripper
- Kali Linux VM
- Windows VM, Ubuntu (22.04 LTS)

Difficulty: Intermediate
Ethical Note: Do **not** attempt any of these techniques on real systems you don't own or have explicit permission to test. All work must be done inside the provided lab VMs and files. The lecturer and the University of Technology, Jamaica will not be liable if you disregard this message and commit a cybercrime.

Definitions:

NTLM (NT LAN MN)

- What it is: A hashing algorithm (challenge/response protocol) used by Windows to store and verify passwords.
- Key point:
    - NTLM hashes are derived from user passwords.
    - They are not salted, which makes them vulnerable to rainbow tables and offline cracking.
- Why it matters in cracking:
    - Once an attacker extracts an NTLM hash, they can attempt to crack it offline or even reuse it in Pass-the-Hash attacks without knowing the actual password.

SAM (Security Account Manager)

- ***What it is:*** A Windows registry hive file that stores local user account information, including password hashes.
- ***Location:***
    - C:\Windows\System32\config\SAM (but locked while Windows is running).
- ***Why you need it:***
    - The SAM file holds the actual NTLM hashes for local accounts.
    - However, these hashes are encrypted and can't be used directly without another piece…

SYSTEM (System Hive)

- ***What it is:*** Another Windows registry hive file, storing configuration info for the OS (like system-wide settings, drivers, etc.).

- ***Location:***
    - C:\Windows\System32\config\SYSTEM
- ***Why you need it:***
    - Inside SYSTEM is the Boot Key (a.k.a. SysKey), which Windows uses to encrypt/decrypt the hashes in the SAM file.
    - Without SYSTEM, the hashes from SAM can't be decrypted properly.

Learning Outcomes

By the end of this lab you will be able to:

1. Explain how Windows NT LAN Manager (NTLM) and Linux password hashing schemes work and why *salts* and *rounds* matter.
2. Extract password hashes safely from Windows (SAM/SYSTEM) and Linux (/etc/passwd + /etc/shadow).
3. Use john and its helpers (zip2john, unshadow) to crack Windows, Linux, and ZIP passwords.
4. Choose effective cracking modes: **single**, **wordlist+rules**, and **incremental/mask**.
5. Recommend concrete hardening steps that materially improve password security.

Lab Setup Checklist

- Kali (or Parrot OS) with John the Ripper installed: john, zip2john, unshadow.
- rockyou.txt wordlist available (Kali: /usr/share/wordlists/rockyou.txt.gz — decompress it).
- A Windows 10 VM (local admin) and a Linux VM (sudo).
- **Snapshot** both VMs before you start.

Pro Tip: Keep a lab notes file. Record commands, timings, cracked accounts, and what finally worked. This makes your debrief (and grading!) much easier.

Section 1 — Cracking Windows Passwords (NTLM)

Mission Brief

You will acquire Windows password hashes from the **SAM** database (with **SYSTEM** hive) and crack them with John. You will try three modes: --single, --wordlist --rules, and --incremental.

Background — What matters

- **NTLM** hashes are not salted. Reuse + short passwords are highly vulnerable to offline cracking.

- Windows stops generating **LM** hashes by default on modern versions (good!).
- Defenders should prefer **length** (passphrases), MFA, and reduce NTLM usage in the domain.

Step-by-Step

A. Extract hashes from Windows (safe, offline method)

1. On the Windows 10 VM (run 'cmd' as **Administrator**) and create the hives to be exported as follows:

```
reg save HKLM\SAM C:\Users\Public\SAM.save
reg save HKLM\SYSTEM C:\Users\Public\SYSTEM.save
```

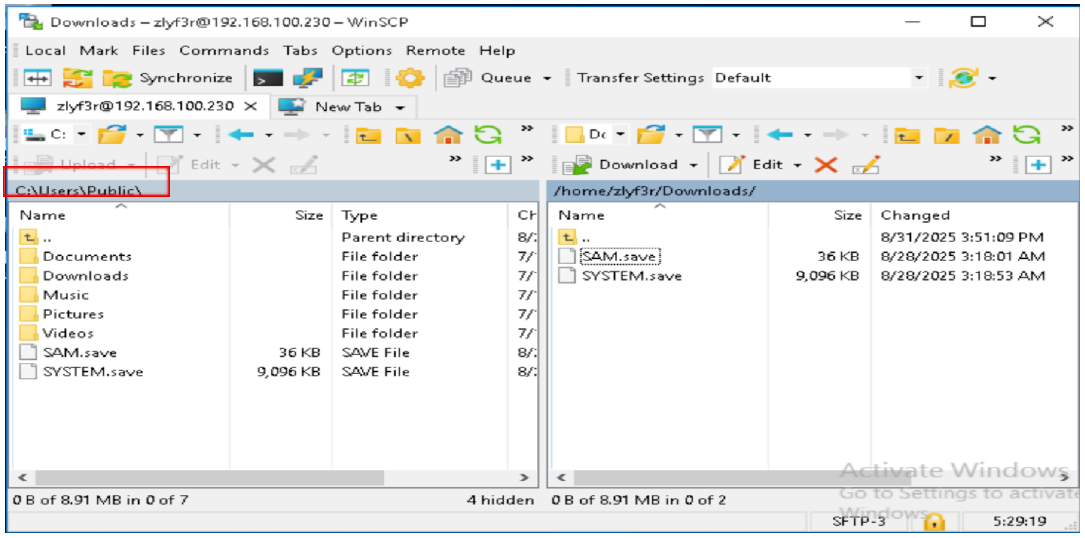
What they do:

- HKLM\SAM → saves the **Security Account Manager (SAM)** hive, which contains user account information and password hashes.
- HKLM\SYSTEM → saves the **SYSTEM** hive, which is needed to decrypt the SAM data.
- The two .save files will be written to C:\Users\Public\, which is convenient to copy out to your Kali VM.

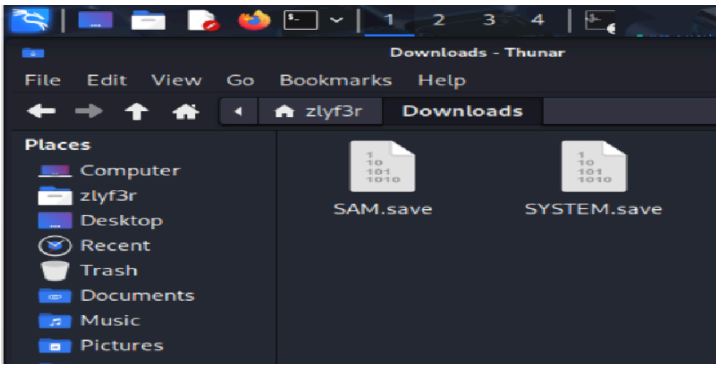
B. Transfer the two .save files to your Kali VM using WinSCP (GUI)

1. Download and install **WinSCP** on the Windows 10 VM.
2. Open WinSCP → create a new session:
    - ***Host:*** your Kali IP (e.g., 192.168.56.101)
    - ***Username:*** your Kali username
    - ***Password:*** your Kali password
    - ***File protocol:*** SFTP
3. Connect and drag-drop the SAM.save and SYSTEM.save files from

   C:\Users\Public\ on Windows into /home/kali/ on Kali.



4. Confirm the files appear in Kali's home directory.



C. Convert hives to NTLM hashes

1. On Kali, use Impacket tool [if not on Kali by default then do the following]
   - `sudo apt update`
   - `sudo apt install python3-impacket`
2. Use Impacket's secretsdump:

```
└─$ impacket-secretsdump -sam SAM.save -system SYSTEM.save LOCAL > winhashes.txt
```

3. Open the winhashes.txt file to see the content using the '***cat***' command

```
└─$ cat winhashes.txt
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Target system bootKey: 0x684ad1e8eb4becc2efe50ed574ea82af
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:37585c3b1142822714d518bca8775616:::
cyberx:1001:aad3b435b51404eeaad3b435b51404ee:32ed87bdb5fdc5e9cba88547376818d4:::
hacker:1002:aad3b435b51404eeaad3b435b51404ee:a4f49c406510bdcab6824ee7c30fd852:::
[*] Cleaning up...
```

D. Crack with John

1. **Single mode** (quick wins using usernames as seeds):

```
john --single --format=NT winhashes.txt
```

2. **Wordlist + rules** (your main attack):

```
john --format=NT --wordlist=/usr/share/wordlists/rockyou.txt --rulewinhashes.txt
```

john –format=NT
cd /usr/share/wordlists
rockyou.txt.gz
gunzip rockyou.txt.gz - rockyou.txt

For example:

```
└─$ sudo john --format=NT --wordlist=/usr/share/wordlists/rockyou.txt --rules winhashes.txt
[sudo] password for fire:
Using default input encoding: UTF-8
Loaded 4 password hashes with no different salts (NT [MD4 256/256 AVX2 8×3])
Remaining 3 password hashes with no different salts
Warning: no OpenMP support for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
123456           (cyberx)
Password         (hacker)
```

3. **Incremental** (brute-force-like, slower):

```
john --format=NT --incremental winhashes.txt
```

4. **Show cracked creds**:

```
john --show --format=NT winhashes.txt
```

Questions

1. Why does the lack of a **salt** make NTLM susceptible to massive precomputed attacks?
2. If you increase password **length** to 16+ chars (passphrases), what happens to attack feasibility?
3. Name two Windows **defensive** controls to reduce NTLM risk in enterprises.
4. What is **Pass-the-Hash**, and how is it different from cracking?

Section 2 — Cracking Linux Passwords (shadow-based)

Mission Brief

You will join **/etc/passwd** and **/etc/shadow** using unshadow, then crack the resulting file. You'll interpret hash identifiers and compare cracking difficulty across schemes.

Background — What matters

- Linux stores password hashes in /etc/shadow, referenced by /etc/passwd.
- Hash ID prefixes: Older less secured hashing algorithm [1 = MD5-crypt, 6 = SHA-512-crypt, $2y$ = bcrypt,] y = yescrypt (modern- most secured).
- **Salt** and **rounds** increase work per guess, dramatically slowing attackers.

Step-by-Step

1. On the Linux VM, create **two test users** (one easy, one strong):

```
sudo adduser alice   # choose an easy lab password like P@ssw0rd123
sudo adduser bob     # choose a long passphrase (e.g., 5+ random words)
```

2. Safely copy account databases for offline cracking:

```
sudo cp /etc/passwd ~/passwd.copy
sudo cp /etc/shadow ~/shadow.copy
sudo chmod 644 ~/passwd.copy ~/shadow.copy
```

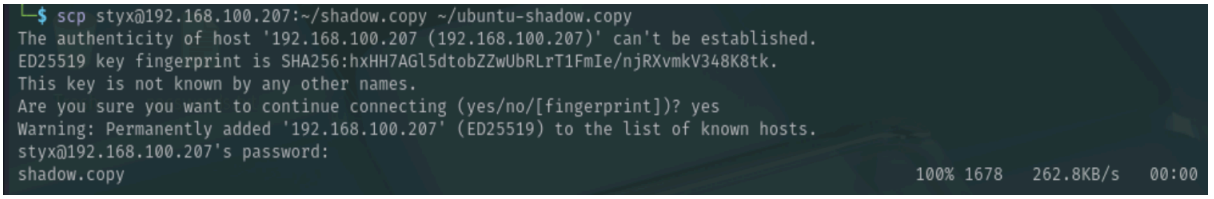
   These files are now readable and can be transferred.

3. Transfer them to Kali:

   Option 1 (SCP command line, from Kali):

```
scp victimuser@<Ubuntu-IP>:~/passwd.copy ~/ubuntu-passwd.copy
scp victimuser@<Ubuntu-IP>:~/shadow.copy ~/ubuntu-shadow.copy
```





   Replace victimuser with your Ubuntu username and <Ubuntu-IP> with the IP from ip addr.

4. Option 2 (WinSCP GUI):
   - Connect to the Ubuntu VM using SFTP (host = <Ubuntu-IP>, port = 22).
   - Drag passwd.copy and shadow.copy into your local machine or directly into Kali's home directory.
5. On Kali, combine them with unshadow

```
unshadow ubuntu-passwd.copy ubuntu-shadow.copy > linux.hashes
```

   Note: /etc/shadow is highly sensitive. Never attempt this outside of your controlled lab VMs.

6. Crack with John — try three modes as before:

```
john --single linux.hashes
john --wordlist=/usr/share/wordlists/rockyou.txt --rules linux.hashes
john --incremental linux.hashes
```

7. Show results:

```
john --show linux.hashes
```

Questions

1. What is a **salt** and how does it defeat precomputed rainbow tables?
2. What do **rounds** (e.g., in SHA-512-crypt) do to an attacker's cost per guess?
3. Why is bcrypt/yescrypt generally more resistant than MD5/SHA-512-crypt at the same hardware budget?

Section 3 — Cracking ZIP Files (ZipCrypto & AES)

Mission Brief

You'll create password-protected ZIPs (legacy ZipCrypto and modern AES) and crack them using zip2john + john. You'll learn why legacy encryption is weak and how strong passwords defeat offline attacks even for AES.

Background — What matters

- **ZipCrypto (PKZIP 2.0)** is weak and often crackable with wordlists quickly.
- **AES-256** (as used by 7-Zip/WinZip) is cryptographically strong; password strength becomes the deciding factor.

Step-by-Step

A. Make a legacy-encrypted ZIP (ZipCrypto)

1. Create a test file and a ZipCrypto archive:
2. echo "Top secret lab note" > secret.txt

zip -e secrets-legacy.zip secret.txt # prompts for password (choose a weak demo one)

3. Extract hash for John:

zip2john secrets-legacy.zip > zip-legacy.hash

4. Crack it:
5. john --wordlist=/usr/share/wordlists/rockyou.txt --rules zip-legacy.hash

john --show zip-legacy.hash

B. Make an AES-encrypted ZIP

1. Use 7-Zip (installed on Kali as 7z) to create AES-256 archive:

7z a -p -mem=AES256 secrets-aes.zip secret.txt # use a strong passphrase here

2. Extract hash for John (same tool works):

zip2john secrets-aes.zip > zip-aes.hash

3. Attempt cracking (likely much harder):

john --wordlist=/usr/share/wordlists/rockyou.txt --rules zip-aes.hash

4. If it doesn't crack in a few minutes, stop and document why — the crypto is strong; a solid passphrase resists offline attacks.

Mini-Questions

1. Why is **ZipCrypto** usually crackable but **AES**-encrypted ZIPs aren't (given strong passwords)?
2. What strategies would you use to guess a human-chosen but *memorable* passphrase?

Blue Team Debrief — Protecting ZIPs

- Prefer **AES-256** archives (7-Zip/modern WinZip).
- Use **long passphrases** (4–6+ random words).
- Don't reuse passwords; rotate sensitive archives.